BDF - Customized Product Documentation

The BDF Bakery Automation System (Racer2 V3) is designed to optimize quality control and production monitoring for your bakery operations. This document provides a professional and simple guide to its features, configuration, and troubleshooting, tailored for BDF's specific needs in automation.

BDF Bakery Automation System: Product Overview

Purpose

The BDF Bakery Automation System (Racer2 V3) is engineered to enhance quality control on your production line. It provides advanced line-scan acquisition, precise frame stitching, live streaming, and optional AI-assisted inspection for bakery products. Designed to integrate with your existing setup, it supports both hardware-backed production and flexible development workflows.

Core Capabilities for Bakery Production

- **Dual-Camera Acquisition & Stitched Frame Generation:** Captures data from two Basler line cameras and seamlessly combines them into a single, comprehensive image of your product for thorough inspection.
- **Browser-Friendly JPEG Capture & Live Streaming:** Provides immediate visual feedback through high-quality JPEG images and real-time video streams, accessible via a web browser for operator monitoring.
- **Flexible Camera Access:** Supports direct `local` camera access, `remote` access to a dedicated camera service, or `emulated` (synthetic) frames for testing without physical cameras.
- **Optional AI-Assisted Inspection:** Integrates advanced AI models (RF-DETR or YOLO) for automated detection and tracking of product features or defects, ensuring consistent quality.
- **Tracked-Object Color Analysis:** Monitors and analyzes color characteristics of specific product features, providing valuable data for quality assurance and recipe consistency at BDF.
- **External & Local Inference:** Supports forwarding frames to external services (`POST /analyze`) or performing AI inference on local image files (`POST /inference`) for diverse inspection needs.

Operating Modes for BDF Operations

The system can be configured to suit different operational needs within BDF's production environment:

- **`local` Camera Mode:** The main application directly manages the Basler cameras. This is ideal for single-host deployments where the processing unit is co-located with the cameras on the production line.
- **`remote` Camera Mode:** The main application communicates with a separate, dedicated camera service over your network. This allows the camera hardware to remain close to the production environment while the AI and web interface can run on a different, potentially more powerful machine.
- **`emulated` Camera Mode:** Generates synthetic frames internally, perfect for development, testing new configurations, or operator training without requiring actual camera hardware on the BDF line.

AI processing can be:

- **Disabled:** For applications requiring only raw streaming and image capture, without automated inspection.
- **Enabled with `rfdetr`:** Provides advanced detection and tracked annotation capabilities.
- **Enabled with YOLO plus SORT:** Offers an alternative detection and tracking solution for specific product types.

High-Level Production Flow

1. The BDF Automation System launches, initializing based on your chosen camera and AI settings.

2. Frames are continuously captured from cameras, fetched from a remote service, or emulated internally.

3. If AI is enabled, models load in the background, making the system ready for intelligent inspection without blocking server startup.

4. The latest frame data is made available for:

- Real-time viewing in the browser UI for operators.
- API access for external systems (e.g., `GET /capture` for integration).
- Automated inspection requests (`POST /analyze`, `POST /inference`).
- Live WebSocket streaming to operator dashboards.

5. With color analysis active, tracked product features are analyzed, and their color data is buffered for quality reporting.

System Architecture for BDF Production Monitoring

Understanding the BDF Bakery Automation System's architecture helps optimize its use for production monitoring and quality control.

Runtime Layers

The system is organized into distinct layers to manage the flow from hardware to user interface:

- **Entrypoints:** Initiates the main application and, if needed, the dedicated camera service.
- `main.py`: Launches the core application, typically on port `8000`.
- `camera_main.py`: Launches the independent camera service, typically on port `8001`.
- **Server Layer:** Exposes the user interface and key data endpoints for BDF operators.
- Handles the main UI, streaming data, AI results, and color analysis.
- Provides the host-side camera API for `remote` camera mode, allowing flexible deployment.
- **Library Layer:** Contains specialized, reusable modules for camera control, AI processing, and color analysis.

Camera Runtime Selection

The system adapts to your BDF production environment by allowing different camera runtime configurations:

- **`local`:** Basler camera access is managed directly within the main application process. This simplifies deployment on a single machine where cameras are physically attached.
- **`remote`:** Frames and alignment data are securely fetched from a separate camera service. This is ideal when cameras need to be physically isolated or on a different host from the main AI/web application, common in complex production lines.
- **`emulated`:** Synthetic frames are generated within the application for development, testing, and UI validation without requiring physical cameras on the BDF line.

The dedicated camera service (on port `8001`) exclusively supports `local` and `emulated` modes, focusing solely on camera management and not hosting AI features.

Camera Production Pipeline

The core live acquisition and frame stitching process ensures high-quality images for BDF's quality control needs. This pipeline operates as follows:

1. **Device Discovery:** Identifies connected Basler cameras.

2. **Configuration:** Sets up cameras for trigger, pixel format, and optional white balance, unless `CAMERA_CONFIG_BY_PYLON=true` (meaning an external tool like Pylon Viewer manages settings).

3. **Acquisition:** Reads individual strips of images from each camera as they pass.

4. **Vertical Stacking:** Stacks these strips vertically for each camera.

5. **Offset Adjustment:** Applies horizontal and vertical offsets for precise alignment between camera views.

6. **Image Stitching:** Combines outputs from multiple cameras into a single, unified image of the product.

7. **Color Conversion:** Converts frames to BGR format if required by the selected pixel format (e.g., from YUV formats) to ensure accurate color representation for AI and display.

8. **Output Preparation:** Resizes and JPEG-encodes the final image for efficient transmission and storage.

9. **Caching:** Stores the latest JPEG for immediate access by HTTP and WebSocket consumers.

Operators can dynamically adjust alignment during production via `GET /camera/alignment` and `POST /camera/alignment` endpoints to maintain optimal image quality.

AI Inspection Pipeline

The AI pipeline is crucial for automated product inspection at BDF:

- **Model Selection:** Controlled by `AI_MODEL_NAME`, allowing choice between `rfdetr` for advanced detection and tracking, or YOLO plus SORT for general detection and tracking, adaptable to specific bakery products.
- **Asynchronous Initialization:** AI model loading happens in the background, ensuring the server starts quickly and operators can access frames while AI prepares for inspection.
- **Status Monitoring:** `/status` provides updates on AI checkpoint progress, informing operators when the AI is ready for precise inspection.
- **Color Analysis Integration:** The AI pipeline incorporates a `ColorBuffer` to store recent color data and history for tracked objects, enabling detailed quality reporting for BDF.

Data Consumers for Production Monitoring

The processed image data is made available to various points for BDF's monitoring needs:

- **`GET /capture`:** Retrieves the latest JPEG frame.
- **`POST /analyze`:** Forwards the latest frame to an external inference service for specialized analysis (e.g., for complex defect detection).
- **`POST /inference`:** Performs local AI inference on images from disk (e.g., for batch inspection of stored images).
- **`WebSocket /ws`:** Provides a live stream for real-time operator viewing and interaction.
- **Browser UI:** Displays the live feed and AI annotations directly in the web interface for immediate feedback.

Configuration & Monitoring Essentials for BDF

Proper configuration ensures the BDF Bakery Automation System operates optimally for your production line. Monitoring endpoints provide critical insights into system health and production quality.

Key Configuration for BDF Production

Configuration settings are managed through `settings.json` (non-secret runtime settings) and `.env` files (secrets and environment selection).

Loading Order (Priority from lowest to highest):

1. `.env`

2. `APP_ENV_FILE` (if set) or `.env.debug` (if `APP_ENV=debug`)

3. `settings.json`

- **Important:** Environment variables override settings in `settings.json`.

Essential `settings.json` Keys for BDF:

- **`CAMERA_MODE`**: Critical for selecting your camera setup:
- `local`: For direct Basler camera access from the main application.
- `remote`: To poll a separate camera service.
- `emulated`: For development and UI testing without cameras.
- **`CAMERA_PIXEL_FORMAT`**: Defines how camera buffers are interpreted (e.g., `BGR8` for OpenCV-ready frames, `YCbCr422_8` for YUV conversion). Ensures correct color representation for quality control.
- **`CAMERA_CONFIG_BY_PYLON`**: Set to `true` if camera features (like pixel format, exposure, trigger) are configured externally (e.g., via Pylon Viewer) and the BDF system should not overwrite them during startup.
- **`REMOTE_CAMERA_BASE_URL`**: Used when `CAMERA_MODE=remote` to specify the network address of your dedicated camera service (e.g., `http://<camera-host>:8001`).
- **`AI_MODEL_NAME`**: Selects the AI model for automated inspection:
- `rfdetr`: For RF-DETR based detection and tracking.
- Any other value: Activates YOLO detection with SORT tracking.
- **`DETECTION_AND_TRACK`**: A boolean setting to enable or disable the AI detection and tracking features, depending on whether automated inspection is required.

`.env` Files for Security

Use `.env` files to store sensitive information specific to BDF operations:

- `ROBOFLOW_API_KEY`, `RFDETR_MODEL_ID`: For Roboflow-backed AI models.
- `INFERENCE_API_KEY`, `INFERENCE_API_HEADER`: For secure access to external inference services.

Recommended Practice: Keep secrets only in `.env` files. Store environment-specific but non-secret settings in `settings.json`.

API Endpoints for Production Monitoring

These endpoints allow BDF operators and automated systems to monitor the production line and retrieve critical data.

- **`GET /status`**: Provides a comprehensive overview of the system's runtime state, including AI initialization, camera readiness, and stream status. Essential for quick diagnostic checks.
- Example: `{ "ai": { "phase": "ready" }, "camera": { "ready": true } }`
- **`GET /capture`**: Retrieves the latest high-quality JPEG frame from the camera cache. Useful for capturing specific product images for manual review or record-keeping.
- *Returns `image/jpeg` on success, `503` if camera not ready.*
- **`POST /analyze`**: Forwards the latest production frame to an external inference service. Ideal for integrating with specialized BDF quality analysis platforms.
- **`GET /camera/alignment` / `POST /camera/alignment`**: Allows retrieval and adjustment of camera stitching alignment settings. Operators can fine-tune image composition for optimal product representation.
- **`GET /color-data`**: Returns buffered color analysis data for tracked objects, then clears the buffer. Provides quantitative color metrics for BDF products.
- **`POST /color-analysis`**: Returns the current color analysis buffer content without clearing it, offering product-shaped color data for downstream consumption.
- **`GET /export_color_csv`**: Exports the current color buffer as a CSV file, useful for trend analysis and quality reporting.

Live Production Stream: `WebSocket /ws`

The WebSocket stream provides real-time visual feedback for operators:

- **Streams:** Either raw JPEG frames or AI-annotated JPEG frames, depending on `DETECTION_AND_TRACK` setting.
- **Status Messages:** Sends text messages like `warming_up` or camera error details during non-frame conditions, keeping operators informed about system state.

Troubleshooting for BDF Production Operators

This section provides guidance for BDF operators to quickly diagnose and resolve common issues, minimizing downtime on the production line.

General Diagnostic Sequence

When an issue arises, follow these steps:

1. **Check System Status:** Call `GET /status` via your browser or API tool.
2. **Verify Camera Readiness:** Confirm `camera.ready` is `true`.
3. **Test Frame Capture:** Check if `GET /capture` successfully returns a JPEG image.
4. **Inspect Browser UI:** Confirm the browser page on `/` updates with live frames.
5. **AI Status (if enabled):** If AI is in use, wait for `ai.phase=ready`.

6. **Test AI/Color Endpoints:** Only then test `POST /analyze`, `POST /inference`, or WebSocket streaming for AI-annotated frames and color data.

Common Production Issues and Solutions

Server Starts But No Frames Arrive

- **Check `/status`:** Verify `camera.ready` is `false` or the WebSocket shows `warming_up`.
- **If using `local` mode:** Ensure Pylon Viewer is closed, both Basler cameras are connected, and the process has camera access.
- **If using `remote` mode:** Verify `REMOTE_CAMERA_BASE_URL` is correct, the camera service is running on the host, and `GET <REMOTE_CAMERA_BASE_URL>/capture` returns a JPEG.
- **If using `emulated` mode:** Confirm `CAMERA_MODE=emulated` and `EMULATED_CAMERA_IMAGE_PATH` points to a readable file if set.

Camera Busy or Unavailable

- **Symptoms:** `/status` reports camera not ready, logs mention camera initialization failure or busy hardware.
- **Solution:** Close Pylon Viewer or any other applications using the cameras. Confirm no other instance of the BDF service is running. A host restart might be necessary if the driver state appears stuck.

Remote Camera Mode Does Not Refresh

- **Check `REMOTE_CAMERA_BASE_URL`:** Confirm no typos and the camera host exposes port `8001`.
- **Network Reachability:** Verify the main app can connect to the camera host from its own environment.
- **Camera Service Health:** Ensure `GET <REMOTE_CAMERA_BASE_URL>/status` returns `200`.

WebSocket Stream Shows Text Instead of Images

- **Expected Behavior:** This occurs when the camera is warming up, initialization failed, or a valid frame hasn't been polled in `remote` mode yet.
- **Solution:** The text typically indicates camera status or `warming_up`. Wait for the camera to become ready.

AI Never Becomes Ready

- **Symptoms:** `/status` shows `ai.phase` stuck in `pending` or `active`.
- **Verification:** Check `/status` for error messages. Ensure your `.env` contains valid Roboflow credentials and `AI_MODEL_NAME` is set correctly. Large models may take time to initialize.

`POST /inference` Returns 404

- **Symptoms:** The endpoint cannot find the specified image file.
- **Verification:** Ensure `SrcDir` points to an existing directory on the machine running the main app, and `ImageName` is correct. The combined path `SrcDir/ImageName` must exist locally. This endpoint does not fetch remote URLs.

Color Endpoints Return Empty Data

- **Possible Reasons:**
- `DETECTION_AND_TRACK=false`: AI detection is required for color analysis.
- AI model is not ready yet.
- No tracked objects have been processed since the last buffer clear.
- A previous call to `GET /color-data` or `POST /color-analysis` already cleared the buffer.

Pixel Format Problems (Wrong Colors, Conversion Errors)

- **Symptoms:** Incorrect colors in output frames, conversion errors during capture, or OpenCV errors involving YUV conversion.
- **Verification:**
- `CAMERA_PIXEL_FORMAT` in `settings.json` must match the actual camera output.
- Use `CAMERA_CONFIG_BY_PYLON=true` if an external tool (e.g., Pylon Viewer) manages camera format.
- `BGR8` is suitable when cameras already output OpenCV-ready frames.

Deployment Options for BDF

The BDF Bakery Automation System can be deployed in several configurations to match your operational needs:

- **Single-Process Local Deployment:** Simplest for direct camera access on one Windows machine running the main application.
- Requires `CAMERA_MODE=local`.
- **Debug or Hardware-Free Deployment:** For testing UI/APIs without cameras.
- Requires `CAMERA_MODE=emulated`.
- **Split Camera and App Deployment:** Ideal for separating camera hardware from the main AI/web application (e.g., for performance or physical isolation).
- **Camera Host:** Runs `camera_main.py` (port `8001`).
- **Main App Host:** Runs `main.py` with `CAMERA_MODE=remote` and `REMOTE_CAMERA_BASE_URL=http://<camera-host>:8001`.
- **Docker for the Main App:** Containerizes the main application, often paired with a Windows host running the camera service.
- `REMOTE_CAMERA_BASE_URL=http://host.docker.internal:8001` (for Docker to reach the host).